

# 复杂迷宫场景下的最短路径规划方法探索与优化结题报告

## 一、研究背景与目标

### 1.1 研究背景

迷宫问题在现代科技领域占据重要地位，其最短路径规划的研究对计算机科学、人工智能和算法设计等领域意义深远，广泛应用于游戏设计、机器人路径规划<sup>[1]</sup>以及网络路由优化等方面，为解决复杂现实问题提供新途径。

### 1.2 研究目标

本研究致力于设计高效路径规划算法解决迷宫最短路径问题，涵盖分析现有算法、选择合适算法并优化、算法实现与性能分析。

## 二、研究方法过程

### 2.1 算法选择

综合权衡效率与准确性，重点研究广度优先搜索（BFS）<sup>[2]</sup>、Dijkstra 算法<sup>[3]</sup>和 A\*算法<sup>[4]</sup>。BFS 适用于简单无权迷宫；Dijkstra 算法用于加权图单源最短路径（边权非负）；A\*算法借助启发函数<sup>[5]</sup>引导搜索<sup>[6]</sup>，在复杂有权迷宫中优势明显，但性能取决于启发函数准确性。

### 2.2 技术路线

- 迷宫表示：采用二维数组表示迷宫，0 表示通道，1 表示墙壁，简单易用利于算法操作。
- 算法实现：着重实现 A\*算法，包括定义启发式函数、管理开放列表和关闭列表、执行节点扩展与路径回溯。
- 路径搜索：运用 A\*算法搜索路径，依据迷宫特性和应用需求调整启发式函数。
- 算法优化：通过调整启发式函数、双向搜索等技术提升算法性能。
- 实验测试：在不同规模和类型迷宫中测试算法，记录路径长度并计算时间，分析算法表现与局限。

## 三、研究成果

### 3.1 算法设计

- 图像预处理
  - 读取迷宫图像，若失败则报错。
  - 将图片通过阈值处理生成二值图像，0 表示障碍物，1 表示可通行区域。
- 算法部分
  - BFS 算法**：从起点开始，按距离层次遍历，利用队列存储待探索节点，标记已访问，找到终点即得路径。
  - Dijkstra 算法**：初始化起点到各点距离为无穷大，起点为 0，用优先队列存储节点。每次取出距离最小的未访问节点更新其邻接节点距离，直到找到终

点或队列为空。

- **A\* 算法：**每个节点记录  $g$ （起点到当前实际代价）、 $h$ （当前到终点估计代价）、 $f$  ( $g + h$ )。从起点开始，用优先队列按  $f$  值存储节点，每次取  $f$  最小的节点扩展，直到找到终点。
- **对比试验：**在多种迷宫图像上运行三种算法，记录搜索时间、路径长度，对比分析性能。

### 3. A\* 优化 - 双向搜索

- 优化 1：双向搜索
  1. 同时从起点和终点进行搜索，各自有开放列表、 $g$ 、 $h$ 、 $f$  值和前驱记录。
  2. 交替从较小开放列表取节点扩展，当两边搜索相遇，合并路径。
- 优化 2：改进启发式算法：
  1. 使用曼哈顿距离作为用启发式函数，作为当前节点到终点的估计代价。

## 3.2 算法源代码及功能解释

---

### Algorithm 1 双向 A\* 路径搜索

---

**Require:** 包含起点（红色）和终点（蓝色）标记的图像

**Ensure:** 从起点到终点的最短路径

```
1: 初始化正向和反向的开放集合、came_from、g_score 和 f_score
2: while 正向开放集合和反向开放集合都不为空 do
3:   if 正向开放集合较小 then
4:     从正向开放集合中弹出当前节点
5:     if 当前节点在反向 came_from 中 then
6:       跳出循环
7:     end if
8:     将当前节点标记为正向已访问
9:     更新邻居节点的得分，并将其加入正向开放集合
10:  else
11:    从反向开放集合中弹出当前节点
12:    if 当前节点在正向 came_from 中 then
13:      跳出循环
14:    end if
15:    将当前节点标记为反向已访问
16:    更新邻居节点的得分，并将其加入反向开放集合
17:  end if
18: end while
19: if 找到路径 then
20:   在交点处合并正向和反向路径
21: else
22:   抛出错误：未找到路径
23: end if
24: 记录路径长度、执行时间，并将路径和障碍物数据保存到输出文件
```

---

我们用伪代码来表示了我们的算法，如 Algorithm 1 所示。该算法使用双向 A\* 路径搜索方法，在起点和终点之间进行路径查找。首先，正向和反向的开放集合（open set）、路径记录表（came\_from）、G 值表（g\_score）和 F 值表（f\_score）被初始化。然后，在主循环中，算法根据正向和反向开放集合的大小选择搜索方向。如果正向集合较小，则从正向集合中弹出当前节点并更新其邻居节点的得分；否则，从反向集合中执行相同的操作。当两个方向的路径相交时，合并路径以获得完整路径。若未找到路径，则返回错误。最终，算法记录路径长度、执行时间，并将路径和障碍物数据保存至输出文件。具体的代码如下所示：

### Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import heapq
import time

# 读取图像文件
def read_image(image_path):
    image = cv2.imread(image_path) # 使用 OpenCV 读取图像
    if image is None:
        raise FileNotFoundError(f'图像未找到: {image_path}') # 检查图像是否成功读取
    return image

# 判断两个颜色是否接近
def is_color_close(color, target, threshold=50):
    # 计算两个颜色的欧氏距离, 并判断是否小于阈值
    return np.linalg.norm(np.array(color) - np.array(target)) < threshold

# 将障碍物像素坐标保存到 Excel 文件中
def save_obstacle_pixel_coordinates_to_excel(image, output_path="zhangai.xlsx"):
    rows, cols = image.shape[:2]
    obstacle_pixels = [] # 存储障碍物像素的坐标

    # 遍历图像中的每个像素
    for i in range(rows):
        for j in range(cols):
            # 检查像素是否为障碍物 (灰度值低且不为红色或蓝色)
            if any(image[i, j] < [200, 200, 200]) and not is_color_close(image[i, j], [0, 0, 255]) and
not is_color_close(image[i, j], [255, 0, 0]):
                obstacle_pixels.append((i, j)) # 将坐标添加到障碍物列表
                image[i, j] = [0, 0, 0] # 将障碍物像素颜色设为黑色

    # 将障碍物坐标存储为 Excel 文件
    df = pd.DataFrame(obstacle_pixels, columns=["Row", "Column"])
    df.to_excel(output_path, index=False)
    print(f'障碍物像素坐标已保存到 {output_path}')

# 启发式函数 (曼哈顿距离)
def heuristic(a, b):
    # 计算两个点的曼哈顿距离, 作为启发式函数值
```

```

return abs(a[0] - b[0]) + abs(a[1] - b[1])

# 检查坐标是否合法 (在边界内, 非障碍物, 未访问过)
def is_valid(x, y, rows, cols, maze, visited):
    return 0 <= x < rows and 0 <= y < cols and maze[x, y] != 0 and not visited[x, y]

# 寻找路径
def find_path(image):
    rows, cols = image.shape[:2]
    start, end = None, None # 初始化起点和终点

    # 寻找起点 (红色) 和终点 (蓝色) 像素位置
    for i in range(rows):
        for j in range(cols):
            if (image[i, j] == [0, 0, 255]).all(): # 红色表示起点
                start = (i, j)
            elif (image[i, j] == [255, 0, 0]).all(): # 蓝色表示终点
                end = (i, j)

    if start is None or end is None:
        raise ValueError("未找到起点或终点")

    # 记录开始时间
    start_time = time.time()

    # 正向搜索初始化
    forward_open_set = []
    heapq.heappush(forward_open_set, (0, start))
    forward_came_from = {start: None}
    forward_g_score = {start: 0}
    forward_f_score = {start: heuristic(start, end)}

    # 反向搜索初始化
    reverse_open_set = []
    heapq.heappush(reverse_open_set, (0, end))
    reverse_came_from = {end: None}
    reverse_g_score = {end: 0}
    reverse_f_score = {end: heuristic(start, end)}

    # 八个方向的移动
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1), (-1, -1), (-1, 1), (1, -1), (1, 1)]
    forward_visited = np.zeros((rows, cols), dtype=bool)

```

```

reverse_visited = np.zeros((rows, cols), dtype=bool)

while forward_open_set and reverse_open_set:
    # 正向搜索步骤
    if len(forward_open_set) <= len(reverse_open_set):
        _, current = heapq.heappop(forward_open_set)

        # 如果找到相交点, 则跳出循环
        if current in reverse_came_from:
            break

        x, y = current
        forward_visited[x, y] = True # 标记为已访问

        # 检查每个方向的邻居
        for dx, dy in directions:
            neighbor = (x + dx, y + dy)
            nx, ny = neighbor

            # 判断邻居是否合法
            if is_valid(nx, ny, rows, cols, image[:, :, 0], forward_visited):
                tentative_g_score = forward_g_score[current] + 1 # 假设相邻节点距离为1
                if neighbor not in forward_g_score or tentative_g_score <
forward_g_score[neighbor]:
                    forward_came_from[neighbor] = current
                    forward_g_score[neighbor] = tentative_g_score
                    forward_f_score[neighbor] = tentative_g_score + heuristic(neighbor, end)
                    # 使用 heapreplace 优化 open_set 的操作
                    if (forward_f_score[neighbor], neighbor) in forward_open_set:
                        forward_open_set.remove((forward_f_score[neighbor], neighbor))
                        heapq.heapify(forward_open_set)
                    heapq.heappush(forward_open_set, (forward_f_score[neighbor], neighbor))
    # 反向搜索步骤
    else:
        _, current = heapq.heappop(reverse_open_set)

        # 如果找到相交点, 则跳出循环
        if current in forward_came_from:
            break

        x, y = current
        reverse_visited[x, y] = True

```

```

# 检查每个方向的邻居
for dx, dy in directions:
    neighbor = (x + dx, y + dy)
    nx, ny = neighbor

# 判断邻居是否合法
if is_valid(nx, ny, rows, cols, image[:, :, 0], reverse_visited):
    tentative_g_score = reverse_g_score[current] + 1 # 假设相邻节点距离为1
    if neighbor not in reverse_g_score or tentative_g_score <
reverse_g_score[neighbor]:
        reverse_came_from[neighbor] = current
        reverse_g_score[neighbor] = tentative_g_score
        reverse_f_score[neighbor] = tentative_g_score + heuristic(start, neighbor)
        if (reverse_f_score[neighbor], neighbor) in reverse_open_set:
            reverse_open_set.remove((reverse_f_score[neighbor], neighbor))
            heapq.heapify(reverse_open_set)
            heapq.heappush(reverse_open_set, (reverse_f_score[neighbor], neighbor))

# 合并路径
if current in forward_came_from:
    path = []
    while current is not None:
        path.append(current)
        current = forward_came_from[current]
    path.reverse()
    current = reverse_came_from[path[-1]]
    while current is not None:
        path.append(current)
        current = reverse_came_from[current]

# 计算路径长度
path_length = len(path)
print(f'路径长度: {path_length}')
else:
    raise ValueError("未找到路径")

# 记录结束时间
end_time = time.time()
print(f'路径搜索耗时: {end_time - start_time} 秒')

# 绘制路径
path_image = image.copy()

```

```

path_pixels = []

for x, y in path:
    original_rgb = image[x, y]
    path_pixels.append((x, y, original_rgb[0], original_rgb[1], original_rgb[2]))
    path_image[x, y] = [0, 255, 0] # 将路径颜色设为绿色

# 将路径上的原始RGB值存储为Excel文件
path_df = pd.DataFrame(path_pixels, columns=["Row", "Column", "R", "G", "B"])
path_df.to_excel("path_pixels.xlsx", index=False)
print("路径上原始RGB值已保存")

return path_image

# 主函数
def main(image_path):
    original_image = read_image(image_path) # 读取原始图像
    save_obstacle_pixel_coordinates_to_excel(original_image, "zhangai.xlsx") # 保存障碍物坐标
    path_image = find_path(original_image) # 寻找路径

```

### 3.3 性能评估

TABLE I  
RESULTS AND COMPARATIVE ANALYSIS OF PATHFINDING ALGORITHMS

| Method         | BFS               | Dijkstra          | A*                | Ours                                |
|----------------|-------------------|-------------------|-------------------|-------------------------------------|
| Inference Time | $0.63s \pm 0.18s$ | $0.71s \pm 0.13s$ | $0.54s \pm 0.15s$ | <b><math>0.05s \pm 0.02s</math></b> |
| Distance       | <b>239</b>        | 240               | 241               | 241                                 |

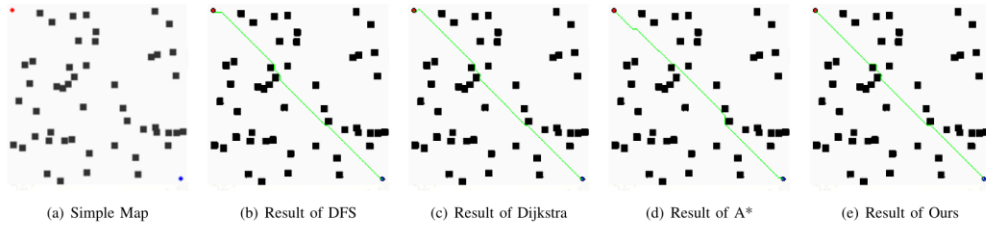


Fig. 1. Illustrations of a simple map and searched paths from path finding methods.

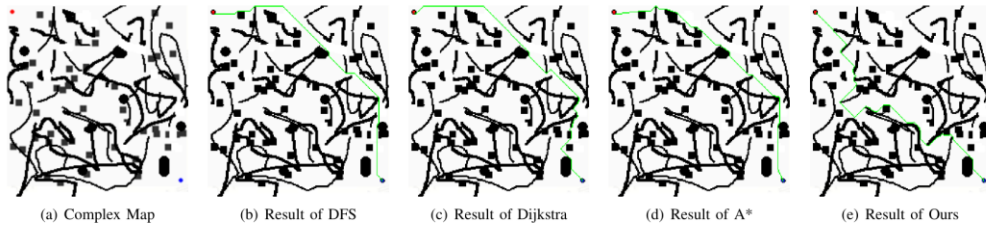


Fig. 2. Illustrations of a more complex map and searched paths from path finding methods.

我们将我们的算法与几个常用的路径搜索方法进行了比较，包括 BFS, Dijkstra, A\* 等。其中我们分别在两张地图中进行测试，包括一张简单的地图，如 Fig1 中的图(a) 所示，以及一张我们人为制作的较为复杂的地图，如 Fig2 中的图(a) 所示，用于测试算法在困难问题中的性能。所有的实验皆在一台配备 i9-13980HX CPU 的笔记本上进行测试。我们在复杂地图下每组实验分别运行 10 次，并测量其平均运行时间，以及搜索到的路径距离。路径结果如 Fig1 和 Fig2，其中复杂地图的具体结果如 Table1 所示。其中，我们在简单地图场景下测试了所有的算法，这些算法都找到了类似的最短路径。在复杂的地图场景下，我们的算法以 **x10 倍以上的速度** 搜索出了与其他方法类似距离的最短路径（最高不超过两个像素点）。这一结果证明了我们的算法具有高效，快速的特点，以及在处理有时间要求的最短路径问题中有巨大的优势。

## 四、研究创新点

### 4.1 算法优化创新

#### 1. 双向 A\*搜索

我们实现了双向 A\*搜索算法，对经典 A\*算法进行优化。在双向搜索中，从起点和终点同时进行路径搜索，最终在两条路径的交汇点相遇。这种方法可以有效减少搜索空间，从而提升算法效率，特别是在大规模地图或复杂迷宫中，我们的实验结果也证明了双向搜索通常比单向搜索更快。

#### 2. 使用曼哈顿距离作为启发式函数

我们设置的移动方向为上下左右四个方向，基于此我们使用了曼哈顿距离作为启发式函数来估算当前节点到终点的距离，以此来精确反映节点之间的距离。此外，曼哈顿距离的计算简单、效率高，尤其适用于网格状的地图。相比于欧几里得距离，曼哈顿距离可以减少计算量，保持 A\*算法的高效性和稳定性。

#### 3. 利用 Python 标准库 `heapq` 实现优先队列：

代码使用了 Python 标准库中的 `heapq` 模块来实现优先队列，从而高效地维护 A\*算法中的 `open\_set`。`heapq` 能够以  $O(\log n)$  的复杂度进行插入和删除操作，确保了 A\*算法在每一步中都能以最小开销选取当前最优节点。这种方法比线性遍历更高效，尤其在路径搜索节点较多的情况下，可以显著降低搜索时间。

### 4.2 迷宫处理创新

在障碍物检测方面，代码使用了颜色阈值和距离判断的组合方法。通过 `is\_color\_close` 函数计算两个颜色之间的欧氏距离，判断障碍物是否与目标颜色接近。这种方法比单纯的 RGB 值比较更加灵活，可以允许一定范围的颜色偏差，增强了障碍物检测的适应性。此外，通过颜色阈值排除了红色和蓝色区域（分别代表起点和终点），确保路径搜索不会将起点或终点误识别为障碍物。



## 参考文献

- [1] L. E. Kavraki, M. N. V. Prasad, and S. M. LaValle. "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566-580, August 1996.
- [2] Sihotang, Jonhariono. "Analysis Of Shortest Path Determination By Utilizing Breadth First Search Algorithm." *Jurnal Info Sains: Informatika dan Sains* 10.2 (2020): 1-5.
- [3] Frana, Philip L., and Thomas J. Misa. "An interview with edsgar w. dijkstra." *Communications of the ACM* 53.8 (2010): 41-47.
- [4] Hong, Zhonghua, et al. "Improved A-Star algorithm for long-distance off-road path planning using terrain data map." *ISPRS International Journal of Geo-Information* 10.11 (2021): 785.
- [5] Lin, Shen, and Brian W. Kernighan. "An effective heuristic algorithm for the traveling-salesman problem." *Operations research* 21.2 (1973): 498-516.
- [6] Ju, Chunyu, Qinghua Luo, and Xiaozhen Yan. "Path planning using an improved a-star algorithm." *2020 11th International Conference on Prognostics and System Health Management (PHM-2020 Jinan)*. IEEE, 2020.